



List of All Functions (by Category) in CODAP

Jump to:

- [Arithmetic Functions](#)
- [Date/Time Functions](#)
- [Lookup Functions](#)
- [Other Functions](#)
- [Statistical Functions](#)
- [String Functions](#)
- [Trigonometric Functions](#)

Function

Description

Example

Arithmetic Functions

`abs(number)`

Computes the absolute value of its argument.

`abs(3 - 5)` is 2

`abs(3-7) = |3 - 7|=4`

`ceil(number)`

Returns the closest integer greater than or equal to its argument.

`ceil(1)` returns 1

`ceil(1.5)` returns 2

`ceil(-1.5)` returns -1

combinations(n, k)	Returns the number of ways that k things can be drawn from a set of n things without regard to order. The formula is $n!/(k!(n-k)!)$.	combinations(5, 3) returns 10 combinations(20, 7) returns 77520
exp(number)	Returns the result of raising e , the base of natural logs to a power.	exp(2) is 7.39, the same as e squared
floor(number)	Returns the largest integer less than or equal to its numeric argument.	floor(1) returns 1 floor(1.5) returns 1 floor(-1.5) returns -2
frac(number)	Returns the fractional or noninteger part of a number.	frac(1.5) returns 0.5 frac(-2.3) returns -0.3
ln(number)	Returns the logarithm of a number to base e .	ln(100) is 4.61
log(number)	Returns the logarithm of a number to base 10.	log(100) is 2.0
pow(number, power)	Returns the result of a number raised to a power.	pow(2,5) is 2^5 , returns 32 pow(10,-3) is $1 / 1000$, or .001
round(number, digits) round(number, places)	Returns the value of its argument rounded to the specified number of decimal places (default 0). This function has an optional, second argument that specifies how many decimals to round to.	round(3.14) is 3 round(π , 4) is 3.1416 round(1276, -2) is 1300
sqrt(number)	Returns the square root of its numeric argument.	sqrt(100) returns 10 sqrt(2) is 1.41 sqrt(-2) returns NaN, meaning Not a Number

<code>trunc(number)</code>	Returns the nearest integer equal to or smaller in magnitude (closer to zero) than its argument.	<code>trunc(5.5)</code> returns 5 <code>trunc(-5.5)</code> returns -5
----------------------------	--	--

Date/Time Functions

<code>date(year, month, day)</code>	Returns a date/time value corresponding to its arguments. If any argument is omitted it is defaulted to 0 (or 1 for the day). If a single large argument is specified, it is assumed to be seconds since January 1, 1970 and is converted accordingly.	<code>date(1969, 7, 16)</code> returns 7/16/1969 <code>date(1395154345)</code> returns 3/18/2014 7:52:25 AM <code>date(2016, 12, 19, 13, 59, 15, 26)</code> returns 12/19/2016 1:59:15 PM <code>date(16)</code> returns 1/1/2016
<code>dayOfMonth(date)</code>	Returns the day of the month corresponding to the given date.	If date is Aug 31, 2005, <code>dayOfMonth(date)</code> returns 31
<code>dayOfWeek(date)</code>	Returns the day of the week (as a number) corresponding to the given date.	If date is Aug 31, 2005, <code>dayOfWeek(date)</code> returns 4 because August 31, 2005 is a Wednesday
<code>dayOfWeekName(date)</code>	Returns the long name of the day of the week corresponding to the given date.	If date is Aug 31, 2005, <code>dayOfWeekName(date)</code> returns "Wednesday"
<code>hours(dateTime)</code>	Returns the hour (in 24-hour time) corresponding to the given date-time.	<code>hours("8-31-2005 8:57 PM")</code> returns 20 (8 PM on a 24-hour clock) <code>hours(date(2017,3,20,7,1,59))</code> returns 7
<code>minutes(dateTime)</code>	Returns the minute of the hour corresponding to the given date_time.	If <code>dateTime</code> is 3:30 PM on Aug 31, 2005, <code>minute(dateTime)</code> returns 30
<code>month(date)</code>	Returns the number of the month corresponding to the given date.	If the date is Aug. 31, 2016, <code>month(date)</code> returns 8
<code>monthName(date)</code>	Returns the long name of the month corresponding to the given date.	If date is Aug 31, 2005, <code>monthName(date)</code> returns "August"

<code>now()</code>	Returns the current date and time.	At the moment this was written, <code>now()</code> returned 12/30/2016 9:58:15 AM
<code>seconds(dateTime)</code>	Returns the second corresponding to the given <code>dateTime</code> .	If <code>date_time</code> is 3:30:05 PM on Aug 31, 2005, <code>second(dateTime)</code> returns 5.
<code>today()</code>	Returns current date with no time.	On the day this was written, <code>today()</code> returned 12/30/2016
<code>year(date)</code>	Returns the year corresponding to the given date.	If <code>date</code> is Aug 31, 2005, <code>year(date)</code> returns 2005

Lookup Functions

<code>first(attribute, filter)</code>	Returns the value of its argument evaluated for the first case.	<code>first(population)</code> returns the first value of the attribute population <code>first(population,sex="female")</code> returns the first value for the attribute population for which the attribute sex is female
<code>last(attribute, filter)</code>	Returns the value of that expression for the last case.	<code>last(amountInBank)</code> returns the last value of the attribute amountInBank <code>last(amountInBank, year>2010)</code> returns the last value for the attribute amountInBank for which the year is greater than 2010
<code>lookupBoundary(boundary_set, boundary_key)</code>	Looks up a boundary, e.g. a country boundary, in an online collection. This boundary can be displayed on a map. Possible values for the <i>boundary_set</i> include <code>country_boundaries</code>, <code>US_state_boundaries</code>, <code>US_country_boundaries</code>, <code>US_congressional_boundaries</code>, and <code>US_puma_boundaries</code>. The <i>boundary_key</i> will vary depending on the <i>boundary_set</i>. For countries use 2 or 3 letter	<code>lookupBoundary(country_boundaries, "Canada")</code> returns the boundary of Canada <code>lookupBoundary(US_state_boundaries, "GA")</code> returns the boundary of the state of Georgia <code>lookupBoundary(US_county_boundaries, "Amador, CA")</code> returns the boundary of Amador county in California <code>lookupBoundary(US_congressional_boundaries, "Kansas 03")</code> returns the boundary of Kansas' 3rd district

country codes. For US states use full state name or USPS abbreviation. For US counties use a geo id as in 'usssccc'. US congressional districts are specified by 'usssdd'. Districts may be specified as the state abbreviation followed by district number as in CO01. CODAP will recognize the LASD (Legal

lookupBoundary(US_puma_boundaries, "G2300100") returns the boundary of the southernmost PUMA in Maine

lookupByIndex("otherDataSetName", "remoteAttributeName", index)

Returns the value of the specified attribute for the case at the specified (1-based) index in the data set name. Note that dataSetName and attributeName must be strings (or expressions that evaluate to strings), while index should evaluate to an integer.

lookupByIndex("United_States", "Full_state_name", index) looks up the attribute Full_state_name in the United_States data set at the index point.

It then returns Full_state_name in your data set.

lookupByKey("otherDataSetName", "returnedAttributeName", "keyAttributeName", keyValue)

Returns the value of the specified returned attribute for the case with the specified keyValue as its value for the keyAttributeName attribute in the specified data set.

lookupByKey("United_States", "State_abbreviations1", "State_abbreviations2", Full_state_name) looks up the states in the data set "United_States."

The attribute "State_abbreviations1" from the first data set matches the attribute "State_abbreviations2" from the second data set.

The function looks up the states "CA," "WA," "OR" and returns "California," "Washington," "Oregon."

next(expression,default, filter)

Returns the value of that expression for the next case.

next(distance) is 10

next(Population, City="Tucson") returns the next value for the attribute Population for which the attribute City is Tucson

prev(expression, default, filter)

Returns the value of that expression for the previous case.

prev(height) returns 9

prev(height, sex="female") returns the previous value for the attribute height for which the attribute sex is female

Other Functions

<code>boolean(expression)</code>	Returns the value of its argument converted to a boolean value.	<code>boolean(age > 60)</code> is true or false <code>age > 60</code> is true or false
<code>greatCircleDistance(lat1, long1, lat2, long2)</code>	Returns the shortest distance between two points on the surface of the earth, in kilometers (km).	<code>greatCircleDistance(40.66, -74, 37.8, -122.4)</code> returns 4128 (the distance in km between New York City and San Francisco)
<code>if(expression, value_if_true, value_if_false)</code>	Returns one of two values depending on whether the result of evaluating a logical expression is true or false.	<code>if(xx>2, "big", "small")</code>
<code>isFinite(expression)</code>	Checks whether a number is a finite number.	<code>isFinite("Dolphin")</code> returns false. <code>isFinite(12)</code> returns true. <code>isFinite(5-2)</code> returns true.
<code>number(expression)</code>	Converts the input value to a numeric value.	<code>number(3.5)</code> returns the number 3.5
<code>random()</code>	Returns a random number drawn from a uniform distribution.	<code>random()</code> returns a number between 0 and 1
<code>random(max)</code>		<code>random(100)</code> returns a number between 0 and 100
<code>random(min, max)</code>		<code>random(-10, 10)</code> returns a number between -10 and 10
<code>randomBinomial(n, p)</code>	Returns a random integer drawn from a binomial distribution with n independent draws (or experiments) each with probability p of success.	<code>randomBinomial(5, 0.5)</code> yields a number of heads in 5 flips of a fair coin <code>randomBinomial(2, 0.1)</code> gives a random number representing the number of red poker chips in two draws from a bag containing 1 red and 9 blue poker chips
<code>randomNormal(mean, standard_deviation)</code>	Returns a random number drawn from a normal distribution which, by default, has a mean of 0 and a standard deviation of 1.	<code>randomNormal()</code> returns a number drawn from a normal distribution with a mean of 0 and a standard deviation of 1

randomNormal(100) returns a number drawn from a normal distribution with a mean of 100 and a standard deviation of 1

randomNormal(-10, 5) returns a number drawn from a normal distribution with a mean of -10 and a standard deviation of 5

randomPick()

Given a list of arguments, chooses one at random and returns it.

randomPick(1,2,3,4,5,6) makes a die, returning one of the number 1–6 chosen at random.

randomPick("heads","tails") makes a fair coin, returning either heads or tails.

randomPick("Male", "Male", "Female") gives you a population to draw from that is two-thirds Male.

randomPick(GirlsName, BoysName) randomly picks either the value of attribute GirlsName or attribute BoysName. For example, it might pick Ismail one time and Joyce another.

string(value)

Returns the value of its argument converted to a string.

string(6) is the string 6

Statistical Functions

correlation(xExpr, yExpr, filter)

Computes Pearson's correlation coefficient for two attributes. The result does not depend on order.

correlation(fathers_height, sons_height) computes the correlation coefficient for the two attributes.

correlation(age, height, age<18) computes the correlation of age and height for those cases that have age less than 18.

count(expression, filter)

Returns the number of cases in the collection. When used with no arguments it returns the number of child cases.

count(NumberOfPets > 0) returns the number of cases for which NumberOfPets is greater than zero.

<code>linRegrSlope(xExpr, yExpr, filter)</code>	Computes the slope of the least squares regression line (lsrl) for two attributes.	<code>linRegrSlope(fathers_height, sons_height)</code> computes the slope of the lsrl for the two attributes. <code>linRegrSlope(age, height, age<18)</code> computes the slope of the lsrl for age and height for those cases that have age less than 18.
<code>linRegrIntercept(xExpr, yExpr, filter)</code>	Computes the y-intercept of the least squares regression line (lsrl) for two attributes.	<code>linRegrIntercept(fathers_height, sons_height)</code> computes the y-intercept of the lsrl for the two attributes. <code>linRegrIntercept(age, height, age<18)</code> computes the y-intercept of the lsrl for age and height for those cases that have age less than 18.
<code>linRegrResidual(xExpr, yExpr, filter)</code>	Returns the difference between the actual response and the predicted response from the least-squares regression line of the response values against the predictor values.	<code>linRegrResidual(horsepower, mpg)</code> returns, for each value of horsepower, the difference between the actual value of mpg and the predicted value.
<code>linRegrPredicted(xExpr, yExpr, filter)</code>	Returns the response predicted by a least-squares regression line of the response values against the predictor values for each predictor value.	<code>linRegrPredicted(horsepower, mpg)</code> returns, for each value of horsepower, the the predicted value of mpg.
<code>max(expression, filter)</code>	Returns the maximum value of its argument evaluated for every case.	<code>max(100)</code> returns 100 <code>max(Height)</code> returns the maximum height
<code>mean(expression, filter)</code>	Returns the arithmetic mean (sum divided by count) of its argument evaluated for every case.	<code>mean(Height)</code> computes the mean of the attribute Height <code>mean(height, Sex = "F")</code> computes the mean height of females
<code>median(expression, filter)</code>	Returns the median. Half the values of the attribute will be above this and half will be below.	<code>median(lifetime)</code> would return the median value for the attribute lifetime
<code>min(expression, filter)</code>	Returns the smallest value in a collection.	<code>min(price)</code> returns the minimum value of the attribute price

min(price, quality = "good") gives the minimum value of price for cases that has quality good

percentile(expression, constant)	Takes two arguments, the first being the attribute for which the percentile will be computed and the second the desired percentile expressed as a decimal fraction between 0 and 1. The second argument should be a constant, not an attribute. If it is an attribute, the value of that attribute for the first case will be used.	percentile(speed, 0.5) is another way to compute the median percentile(score, 0.95) will return the score corresponding to the 95th percentile.
rSquared(xExpr, yExpr, filter)	Computes square of the correlation coefficient for two attributes. The result does not depend on order.	rSquared(fathers_height, sons_height) computes the square of the correlation coefficient for the two attributes. rSquared(age, height, age<18) computes the rSquared value for age and height for those cases that have age less than 18.
stdDev(experssion)	Computes the sample standard deviation.	stdDev(pressure) computes the sample standard deviation of the attribute <i>pressure</i>
sum(expression, filter)	Returns the sum of its argument evaluated for every case.	sum(grade_point) computes the sum of the grade points sum(Mass, Height>60) returns the sum of the masses for heights taller than 60
uniqueValues(expression, filter)	Returns the number of unique values produced by the expression.	uniqueValues(State) returns the number of unique values of the attribute State. If all state names are present, the result would likely be 50. uniqueValues(Age, Sex="male") returns the number of different ages for the cases whose value of <i>Sex</i> is "male".
variance(expression)	Computes the variance of an attribute, that is, the square of the standard deviation.	variance(before-after) computes the variance of the difference of the two attributes <i>before</i> and <i>after</i> .

String Functions

`beginsWith(stringToLookIn, stringToFind)`

Takes two arguments and returns true if the first begins with the second.

`beginsWith(LastName, "Mc")` will return true for "McBride" and false for "Binker"

`charAt(stringToLookIn, index)`

Returns the character at the specified position.

`combine(value)`

Returns the aggregated concatenation of its evaluated argument values.

`combine(flip)` might return "HHT" when there are three cases each with value "H" or "T"

`combine(1)` returns 1, repeated for the number of cases in the case table

`concat(string1, string2, ... string10)`

Takes up to ten arguments and returns a string. The arguments can be numeric or strings. *Note:* You have to include an argument for the space to get a space in the caption.

if a case has the value "Denise" for the attribute *Name* and the value "likes dogs" for the attribute *Hobby*, then `concat(Name, " ", Hobby)` returns "Denise likes dogs."

`endsWith(stringToLookIn, stringToFind)`

Takes two arguments and returns true if the first ends with the second.

`endsWith(LastName, "er")` returns true for "Binker" and false for "McBride"

`findString(stringToLookIn, stringToFind, index)`

Takes three arguments, returning the position of `stringToFind` in `stringToFind` starting from `start`. The first character of `stringToLookIn` is numbered 1. Returns 0 if `stringToFind` is not found. The third argument need not be present and defaults to 1.

`findString("mathematics", "the")` returns 1, but `findString("mathematics", "the", 2)` returns 6

`includes(stringToLookIn, stringToFind)`

Takes two arguments and returns true if the second argument is a substring of the first (also treated as a string).

`includes("the", "he")` returns true; `includes("dancing", "joy")` returns false; `includes(1234, 23)` returns true

`join(delimiter, string1, string2,...,string99 )`

Returns the string formed by concatenating its string arguments, each separated by its separator argument.

`join("/", m, d, y)` returns a date in format "m/d/y"

patternMatches(string_to_look_in, pattern)	Returns the number of times in <i>string_to_look_in</i> the string or regular expression represented by <i>pattern</i> occurs. The search is case insensitive.	patternMatches("How many pickles can Peter Piper pick?", "pick") returns 2 patternMatches("June 25, 2020", "[0-9]") returns 6 because there are 6 digits patternMatches("From June to July", "june july") returns 2
repeatString(aString, numRepetitions)	Takes two arguments, the first a string and the second an integer ≥ 0. Returns the result of concatenating aString numRepetitions times.	repeatString("#&", 4) returns "#&#&#&#&#&"
replaceChars(aString, start, numChars, substituteString)	Takes four arguments. The first is the original string. The second is an integer > 0 specifying the starting location for the substitution. The third is the number of characters to be replaced, and the last is the string that is to be substituted. If numChars is 0, substituteString is inserted.	replaceChars("computer", 3, 4, "nfus") gives "confuser".
replaceString(aString, stringToFind, substituteString)	Takes three string arguments and substitutes the third for all occurrences of the second in the first.	replaceString("12:30:45", ":", " and ") returns "12 and 30 and 45"
sortItems(list, delimiters)	Sorts the items in the list.	sortItems("2,16,3,4,3") returns "2,3,3,4,16" when comma is the regional list separator sortItems("{T;H;12;2;t;h}", "{;}") returns "{2;12;H;h;T;t}" sortItems("HTHHT", "") returns "HHHTT", because all characters are items when there are no delimiters sortItems(FirstName, caseIndex) returns an alphabetized list of first names
split(aString, separator, index)	Returns the string formed by splitting a string by the specified separator and then returning the element at the specified index. Given an attribute "d" that contains strings of the form "mm/dd/yyyy", split(d,"/"),1)	split(09/01/2016, "/", 1) returns September

returns the month, split(d,"/",2)
returns the day, and split(d,"/",3)
returns the year.

stringLength(string)	Returns the number of characters in the string representation of the argument.	stringLength("Fathom") gives 6. stringLength(321) gives 3
subString(string, position, length)	Returns a substring of the specified string, determined by its position and length arguments.	substring(abcd,2,3) returns bcd
toLowerCase(string)	Converts upper-case characters in its argument to lower-case.	toLowerCase(String) returns string
toUpperCase(string)	Converts lower-case characters in its argument to upper-case.	toUpperCase(String) returns STRING
trim(string)	Removes leading, trailing, and redundant white space characters from its argument. (e.g. double spaces are converted to single spaces).	
wordListMatches(string_to_look_in, "datasetName", "wordListAttributeName", "ratingsAttributeName" (optional))	Returns the total number of times any of the words in a given word list is found in the given strToLookIn. If the (optional) name of a numeric rating attribute is specified, the sum of the ratings for the found words is returned. Note that the search is performed with case insensitivity. If an entry in the word list begins and ends with "/" it is treated as a regular expression and searched for with case sensitivity.	Assume a dataset named "Short Words" has an attribute named "word" whose values are "a", "the", and "on". wordListMatches("a party on a Monday", "Short Words", "word") returns 3 because "a" occurs twice, "the" does not occur, and "on" occurs once. Suppose the "Short Words" dataset also has an attribute named "rating" whose are 4, 2, and 5. wordListMatches("a party on a Monday", "Short Words", "word", "rating") returns 13 as the sum of the ratings for each word each time it is found. Suppose the "word" attribute has one value which is "/a/". Then wordListMatches("a party on a Monday", "Short Words", "word") returns 4 since there are 4 occurrences of "a".

Trigonometric Functions

<code>acos(radians)</code>	Returns the angle in radians that corresponds to the specified arccosine value $[-1, 1]$.	<code>acos(1)</code> returns 0 radians
<code>asin(radians)</code>	Returns the angle in radians that corresponds to the specified arcsine value $[-1, 1]$.	<code>asin(1)</code> returns 1.57, $\pi/2$ radians
<code>atan(radians)</code>	Returns the angle in radians that corresponds to the specified arctangent value $[-1, 1]$.	<code>atan(1)</code> returns 0.79, $\pi/4$ radians
<code>atan2(x, y)</code>	Returns the angle in radians between the x-axis and the specified coordinate point.	<code>atan2(1,1)</code> returns 0.79, $\pi/4$ radians
<code>cos(radians)</code>	Returns the angle in radians that corresponds to the specified cosine value $[-1, 1]$.	<code>cos(0.5)</code> returns 0.88 radians
<code>sin(radians)</code>	Returns the angle in radians that corresponds to the specified sine value $[-1, 1]$.	<code>sin($\pi/2$)</code> returns 1
<code>tan(radians)</code>	Returns the angle in radians that corresponds to the specified tangent value $[-1, 1]$.	<code>tan($\pi/4$)</code> returns 1

Related Articles

-  [List of All Functions \(Alphabetical\) in CODAP](#)
-  [Color Points on a Graph by Attribute Values](#)
-  [Enter a Formula for an Attribute](#)

Other Resources

-  [FAQs](#)

GET STARTED

[Tutorials](#)
[CODAP Basics](#)
[Curriculum Examples](#)
[Dataset Examples](#)
[Other Resources](#)

EDUCATORS

[Dataset Examples](#)
[Curriculum Examples](#)

DEVELOPERS

[CODAP Repository](#)
[Plugins](#)
[Plugin Development](#)

RESEARCHERS

[CODAP Publications](#)

COMMUNITY

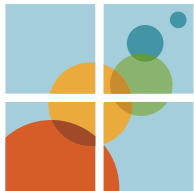
[Forum](#)
[Meetups and Webinars](#)
[Get Involved](#)

HELP

[Forum](#)
[How-to Guides](#)
[FAQs](#)

ABOUT

[Privacy Policy](#)
[Terms of Use](#)



CODAP

A product of  **The Concord Consortium**



©2025 The Concord Consortium